

Persistência com BAAS Supabase

Material
MOBILE_008





BookApp - CRUD
com Expo e
Supabase

➤ O Objetivo

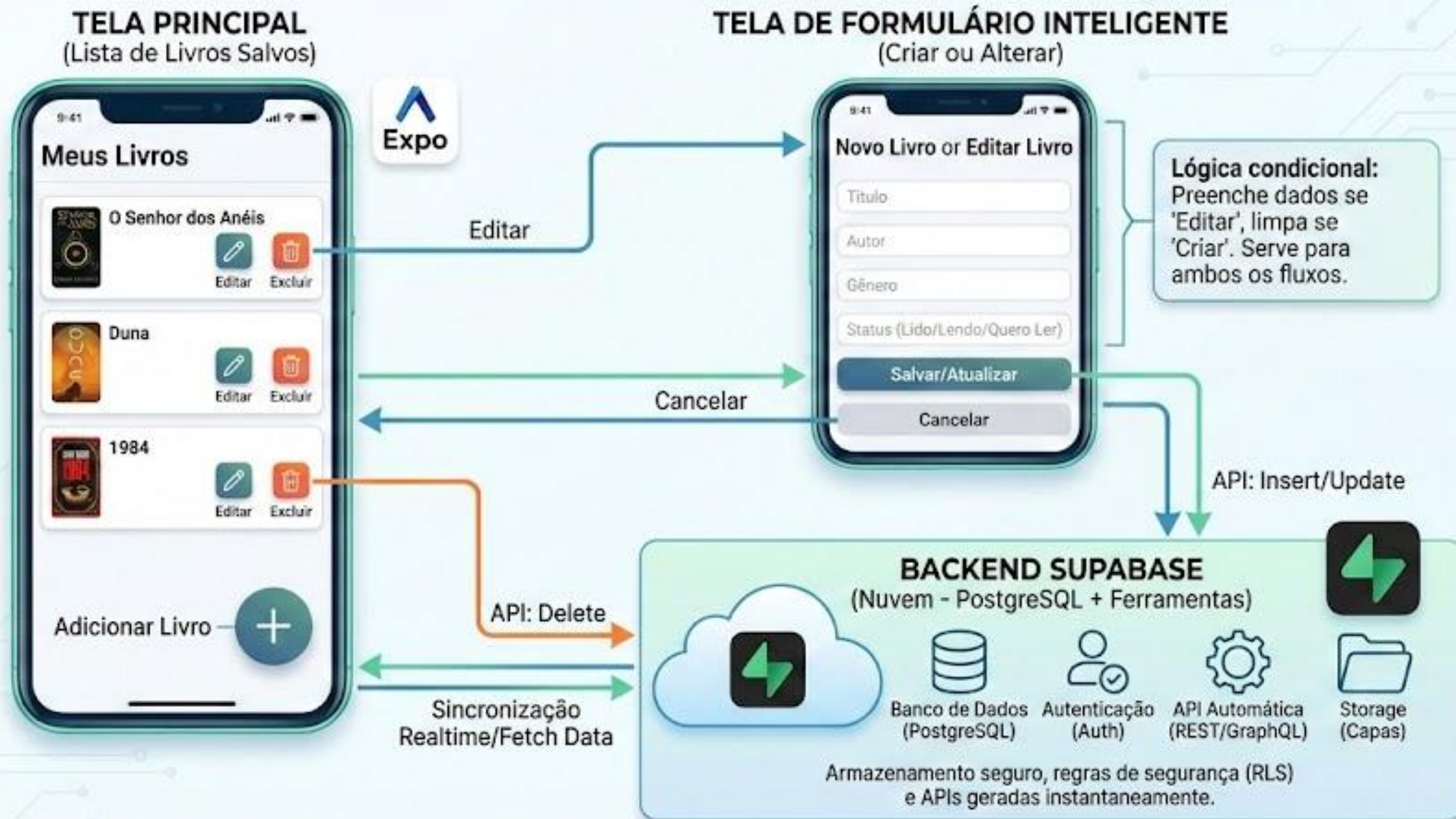
- Criar um aplicativo para gerenciar uma biblioteca pessoal.
 - **Tela Principal:** Lista os livros salvos na nuvem. Cada livro tem botões para **Editar** e **Excluir**.
 - **Botão de Adicionar:** Leva para um formulário.
 - **Tela de Formulário:** Uma tela inteligente que serve tanto para **Criar** um novo livro quanto para **Alterar** um existente.

Porque o supabase

- O **Supabase** é uma plataforma de Backend-as-a-Service (BaaS) que se posiciona, principalmente, como a **alternativa Open Source ao Google Firebase**.
- A grande diferença — e o seu maior trunfo — é que o Supabase é construído inteiramente em cima do **PostgreSQL**, um dos bancos de dados relacionais mais robustos e populares do mundo, em vez de um banco NoSQL proprietário.
- Principais funcionalidades: autenticação, armazenamento, realtime, edge functions e vector (IA).

O Objetivo: Gerenciar uma Biblioteca Pessoal com BookApp (Expo + Supabase)

BookApp: Organização simples e inteligente da sua leitura na nuvem.



Preparação do Ambiente

- Antes de codar, precisamos do banco de dados pronto.
- Acesse supabase.com e crie um projeto novo (BookApp).
- No menu lateral, vá em **Table Editor** e crie uma tabela chamada books.
- Adicione as colunas:
 - id (int8, Primary Key, Identity/Auto Increment)
 - title (text)
 - author (text)
 - created_at (timestampz, default: now())

Table Editor

Update table

books

Name

books

Description

Tabela de Livros

☒

Enable Row Level Security (RLS)

RECOMMENDED

Restrict access to your table by enabling RLS and writing Postgres policies.

i

Policies are required to query data
You need to create an access policy before you can query data from this table.
Without a policy, querying this table will return an empty array of results.

Documentation

☐

Enable Realtime

Broadcast changes on this table to authorized subscribers

Columns

About data type

Name ?	Type	Default Value ?	Primary
id	# int8	NULL	☒
title	T text	NULL	☐
author	T text	NULL	☐
created_at	timestamp	now()	☐

Nota importante

- O RLS ativado (é uma boa prática). Agora você precisa criar uma "regra" dizendo que todo mundo pode mexer nessa tabela:
- Vá em **Authentication** (ícone do cadeado no menu lateral) > **Policies**.
- Encontre a tabela books.
- Clique em **Create Policy**.
- Escolha "**Get started quickly**" e selecione o template:
 - "**Enable read/write access for all**" (Habilitar leitura/escrita para todos). [Próximo Slide]
- Agora clique em **Save policy**.

Create a new Row Level Security Policy

Create a new Row Level Security policy

Policy Name

Enable read access for all users

Table on clause

public.books

Policy Behavior as clause

Permissive

Policy Command for clause

☐ SELECT

☐ INSERT

☐ UPDATE

☐ DELETE

☒ ALL

Target Roles to clause

Defaults to all (public) roles if none selected

USE OPTIONS ABOVE TO EDIT

```
1 create policy "Enable read access for all users"
2 on "public"."books"
3 as PERMISSIVE
4 for ALL
5 to public
6 using (
7   true
8 ) with check (
9   -- Provide a SQL expression for the with check statement
10 );
```

☒ Use check expression

Templates

Search templates

SELECT

Enable read a

This policy giv

all users via th

INSERT

Enable insert

This policy giv

for all authent

DELETE

Enable delete

This policy ass

column "user_

rows which th

their ID

INSERT

Enable insert

This policy ass

column "user_

rows which th

their ID

UPDATE

Policy with tal

Query across

PI S rules

Preparação do App - Frontend

► No terminal do projeto

1. Criar o projeto

```
npx create-expo-app BookApp -t blank
```

```
cd BookApp
```

2. Instalar Navegação + Ícones (para os botões de editar/excluir

```
npm install @react-navigation/native @react-navigation/native-stack react-native-vector-icons  
--legacy-peer-deps
```

3. Dependências do Expo para navegação

```
npx expo install react-native-screens react-native-safe-area-context
```

4. Instalar Cliente Supabase

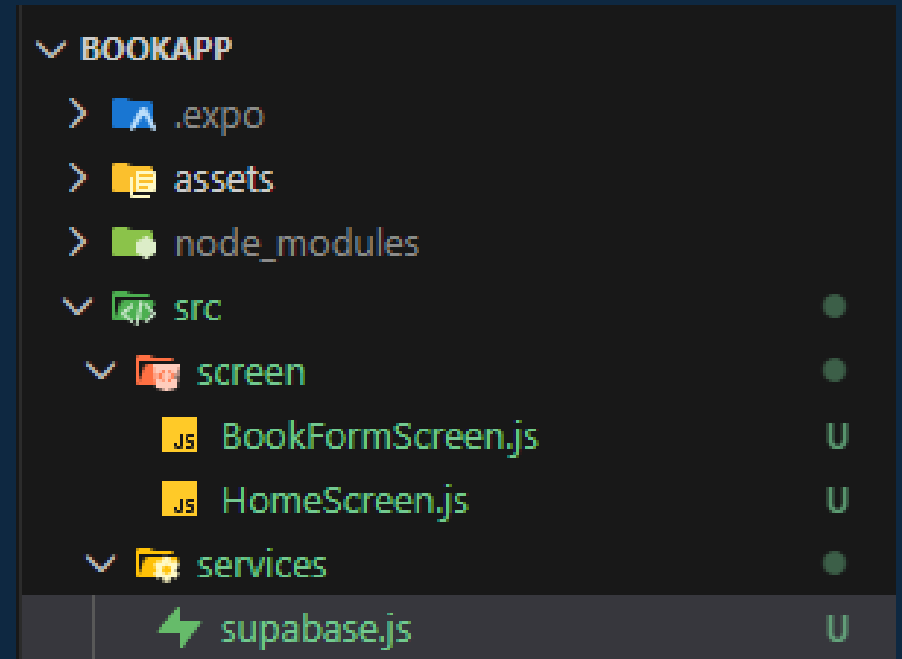
```
npm install @supabase/supabase-js react-native-url-polyfill
```

5. Instalar gerenciador de variáveis de ambiente (Segurança)

```
npm install react-native-dotenv
```

Crie 03 arquivos principais

- 1. src/services/supabase.js (Conexão)
- 2. src/screen/HomeScreen.js (Lista + Excluir)
- 3. src/screen/BookFormScreen.js (Criar + Editar_



Passo 1 – Configurar conexão

Arquivo: src/services/supabase.js

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

// Substitua pelas suas chaves do painel do Supabase (Project Settings > API)
const supabaseUrl = 'SUA_URL_DO_SUPABASE';
const supabaseKey = 'SUA_CHAVE_ANON_PUBLICA';

export const supabase = createClient(supabaseUrl, supabaseKey);
```

API Keys

Configure API keys to securely control access to your project

Publishable and secret API keys Legacy anon, service_role API keys

anon public

eyJhbGciOiJIUzI1NiIsInR5cCI6I

This key is safe to use in a browser if you
Prefer using Publishable API keys instead

service_role **secret**

**** * * * *

This key has the ability to bypass Row Level Security
immediately. Prefer using Secret API keys

Na dúvida, assista:

<https://www.youtube.com/watch?v=EAs3PRquQtw>

“

Passo 2: Tela de Formulário (Criar e Editar)

Esta tela é "inteligente". Se receber um livro via parâmetro, ela entra em modo "Edição". Se não, modo "Criação".

”

Próximos Slides. Explicação do Professor.



```
// src/screen/BookFormScreen.js
import React, { useState, useEffect } from 'react';
import { View, TextInput, Button, StyleSheet, Text, Alert } from 'react-native';
import { supabase } from '../services/supabase';

export default function BookFormScreen({ route, navigation }) {
  // Verifica se veio algum livro para edição (parâmetros da navegação)
  const livroParaEditar = route.params?.book;

  const [title, setTitle] = useState('');
  const [author, setAuthor] = useState('');
  const [loading, setLoading] = useState(false);

  // Se for edição, preenche os campos ao abrir a tela
  useEffect(() => {
    if (livroParaEditar) {
      setTitle(livroParaEditar.title);
      setAuthor(livroParaEditar.author);
    }
  }, [livroParaEditar]);
```



```
const handleSave = async () => {  
  if (!title || !author) {  
    Alert.alert('Erro', 'Preencha todos os campos!');  
    return;  
  }  
  
  setLoading(true);  
  let error;  
  
  if (livroParaEditar) {  
    // --- UPDATE (ATUALIZAR) ---  
    const response = await supabase  
      .from('books')  
      .update({ title, author })  
      .eq('id', livroParaEditar.id); // Onde ID for igual ao do livro  
    error = response.error;  
  } else {  
    // --- CREATE (CRIAR) ---  
    const response = await supabase  
      .from('books')  
      .insert([{ title, author }]);  
    error = response.error;  
  }  
}
```



```
setLoading(false);
```

```
if (error) {  
  Alert.alert('Erro', error.message);  
} else {  
  // Volta para a Home e atualiza a lista  
  navigation.goBack();  
}
```

```
};
```

```
return (  
  <View style={styles.container}>  
    <Text style={styles.label}>Título do Livro</Text>  
    <TextInput  
      style={styles.input}  
      value={title}  
      onChangeText={setTitle}  
      placeholder="Ex: Dom Casmurro"  
    />  
  </View>  
)
```



```
<Text style={styles.label}>Autor</Text>
  <TextInput
    style={styles.input}
    value={author}
    onChangeText={setAuthor}
    placeholder="Ex: Machado de Assis"
  />

  <Button
    title={loading ? "Salvando..." : "Salvar Livro"}
    onPress={handleSave}
    disabled={loading}
  />
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  label: { fontSize: 16, marginBottom: 5, fontWeight: 'bold' },
  input: {
    borderWidth: 1, borderColor: '#ddd', padding: 10,
    borderRadius: 5, marginBottom: 20, fontSize: 16
  }
});
```



Código compartilhado
Link abaixo:

<https://drive.google.com/file/d/1Bo2NIN5v5vapFfsg-94sKYO-8SIKFdd/view?usp=sharing>

“

Passo 3: Tela Principal (Listar e Excluir)

Aqui usamos o hook `useFocusEffect` do React Navigation. Por quê? Porque quando voltamos da tela de "Adicionar Livro", o `useEffect` normal não roda de novo. O `useFocusEffect` garante que a lista recarregue sempre que a tela ganhar foco.

”

Próximos Slides. Explicação do Professor.



```
// src/screen/HomeScreen.js
import React, { useState, useCallback } from 'react';
import { View, Text, FlatList, StyleSheet, TouchableOpacity, Alert, ActivityIndicator } from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { Ionicons } from '@expo/vector-icons'; // Ícones prontos do Expo
import { supabase } from '../services/supabase';

export default function HomeScreen({ navigation }) {
  const [books, setBooks] = useState([]);
  const [loading, setLoading] = useState(true);

  // --- READ (LER) ---
  const fetchBooks = async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('books')
      .select('*')
      .order('id', { ascending: false }); // Mais recentes primeiro

    if (error) Alert.alert("Erro", error.message);
    else setBooks(data);

    setLoading(false);
  };
}
```



```
// Recarrega a lista toda vez que voltamos para esta tela
useFocusEffect(
  useCallback(() => {
    fetchBooks();
  }, [])
);

// --- DELETE (EXCLUIR) ---
const handleDelete = (id) => {
  Alert.alert("Excluir", "Tem certeza que deseja apagar este livro?", [
    { text: "Cancelar", style: "cancel" },
    {
      text: "Apagar",
      style: "destructive",
      onPress: async () => {
        const { error } = await supabase.from('books').delete().eq('id', id);
        if (error) Alert.alert("Erro", error.message);
        else fetchBooks(); // Atualiza a lista após excluir
      }
    }
  ], []);
};
```



```
const renderBookItem = ({ item }) => (  
  <View style={styles.card}>  
    <View style={styles.textContainer}>  
      <Text style={styles.title}>{item.title}</Text>  
      <Text style={styles.author}>{item.author}</Text>  
    </View>  
  
    <View style={styles.actions}>  
      { /* Botão Editar: Navega para o form passando o livro atual */ }  
      <TouchableOpacity  
        onPress={() => navigation.navigate('BookForm', { book: item })}  
        style={styles.actionButton}  
      >  
        <Ionicons name="pencil" size={24} color="#4A90E2" />  
      </TouchableOpacity>  
  
      { /* Botão Excluir */ }  
      <TouchableOpacity  
        onPress={() => handleDelete(item.id)}  
        style={styles.actionButton}  
      >  
        <Ionicons name="trash" size={24} color="#D0021B" />  
      </TouchableOpacity>  
    </View>  
  </View>  
>);
```



```
return (  
  <View style={styles.container}>  
    {loading ? (  
      <ActivityIndicator size="large" color="#000" style={{marginTop: 20}} />  
    ) : (  
      <FlatList  
        data={books}  
        keyExtractor={(item) => item.id.toString()}  
        renderItem={renderBookItem}  
        contentContainerStyle={{ paddingBottom: 80 }} // Espaço para o botão flutuante  
        ListEmptyComponent={<Text style={styles.emptyText}>Nenhum livro cadastrado.</Text>}  
      />  
    )}  
  
    {/* Botão Flutuante (FAB) para Adicionar */}  
    <TouchableOpacity  
      style={styles.fab}  
      onPress={() => navigation.navigate('BookForm')}  
    >  
      <Icons name="add" size={30} color="#fff" />  
    </TouchableOpacity>  
  </View>  
);  
}
```



```
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5', padding: 10 },
  card: {
    flexDirection: 'row', backgroundColor: '#fff', padding: 15,
    borderRadius: 8, marginBottom: 10, alignItems: 'center', elevation: 2
  },
  textContainer: { flex: 1 },
  title: { fontSize: 18, fontWeight: 'bold', color: '#333' },
  author: { fontSize: 14, color: '#666' },
  actions: { flexDirection: 'row', gap: 15 },
  emptyText: { textAlign: 'center', marginTop: 50, color: '#999' },
  fab: {
    position: 'absolute', bottom: 20, right: 20,
    backgroundColor: '#007AFF', width: 56, height: 56,
    borderRadius: 28, justifyContent: 'center', alignItems: 'center', elevation: 5
  }
});
```



Código compartilhado
Link abaixo:

<https://drive.google.com/file/d/1F2D6aAQyyrMKa3XA-h6NV1Ps5-NZMkFY/view?usp=sharing>

“

Passo 4: Juntar tudo no App.js

”

Próximos Slides. Explicação do Professor.



```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './src/screen/HomeScreen';
import BookFormScreen from './src/screen/BookFormScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Minha Biblioteca' }}
        />
        <Stack.Screen
          name="BookForm"
          component={BookFormScreen}
          options={({ route }) => ({
            // Muda o título dinamicamente: "Editar Livro" ou "Novo Livro"
            title: route.params?.book ? 'Editar Livro' : 'Novo Livro'
          })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```



Código compartilhado
Link abaixo:

https://drive.google.com/file/d/1WiPdMDifTSi2gJgyZ6kavqjLzuBm_TzJ/view?usp=sharing

Resumo dos novos Conceitos

- 1. Reutilização de Tela
- 2. `useFocusEffect`
- 3. Botão Flutuante (FAB)
- 4. Ícones



Reutilização de Tela (BookForm)

- Em vez de criar uma tela CreateScreen e outra EditScreen, usamos a mesma.
- A lógica é: "Se veio um ID, é Update. Se não veio, é Insert". Isso é uma prática comum de mercado.

2. useEffect

- No app anterior, carregávamos dados apenas uma vez. Agora, como editamos e voltamos para a lista, precisamos garantir que a lista atualize (re-fetch) sempre que a tela aparecer.

3. Botão Flutuante (FAB)

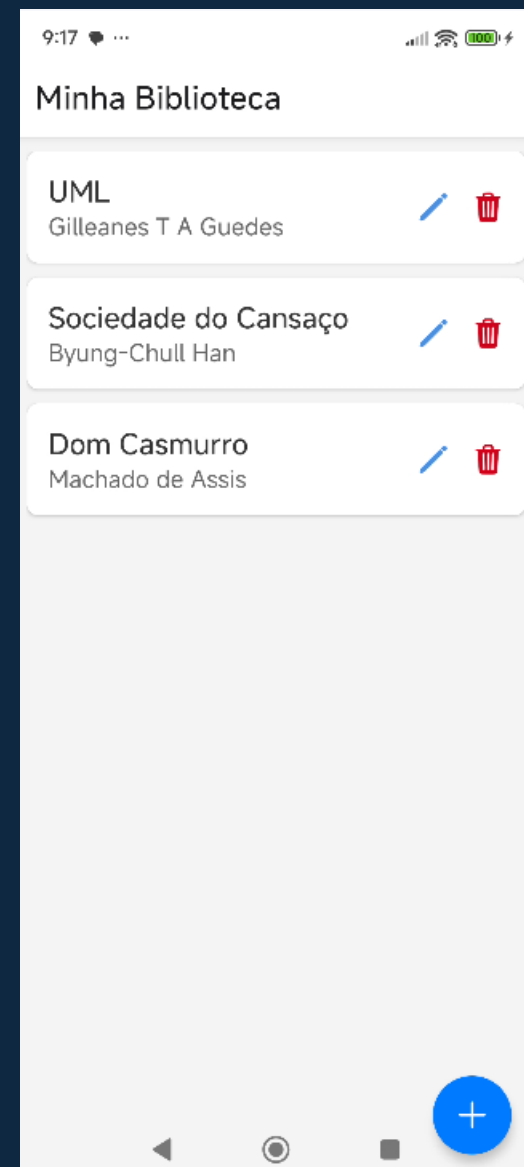
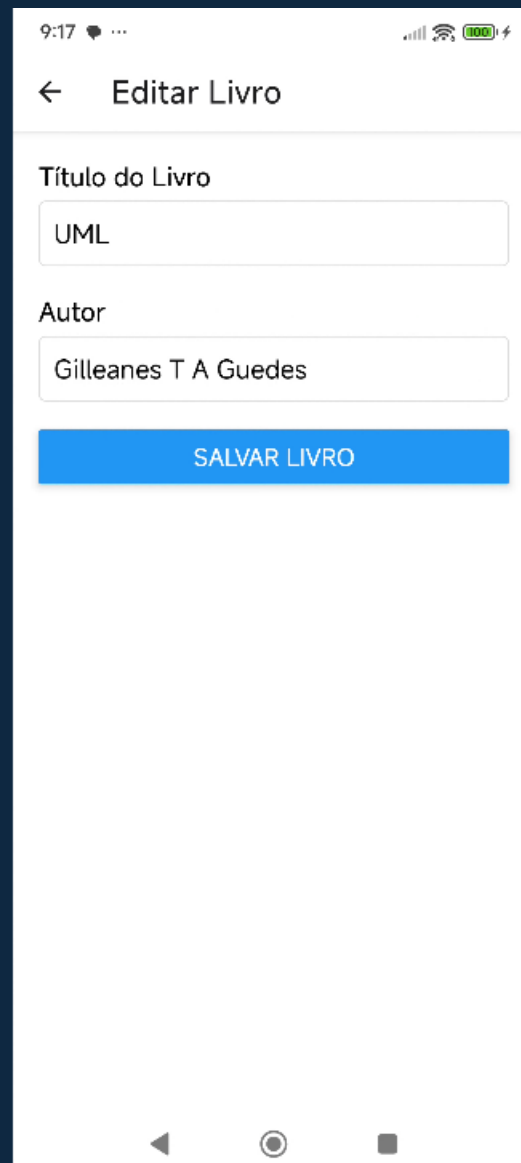
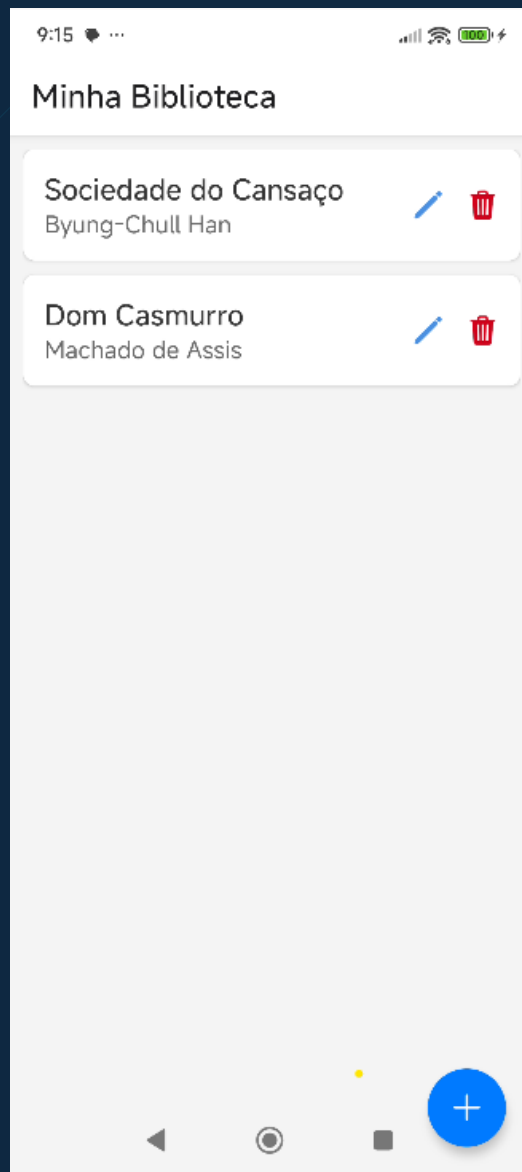
- Implementamos um botão estilo Google (bolinha no canto inferior) usando position: 'absolute', comum em apps de produtividade.

4. Ícones (Ionicons)

- Usamos a biblioteca padrão do Expo para botões visuais (lápiz e lixeira), melhorando a interface.

Agora é só rodar `npx expo start` e testar seu CRUD completo na nuvem!

Resultado



No supabase

santaluciaBookApp / main **PRODUCTION** Connect

books +

Filter Sort Insert

	id int8	title text	author text	created_at timestamptz	
☐	2	Dom Casmurro	Machado de Assis	2025-12-18 16:49:10.010043+00	
☐	3	Sociedade do Cansaço	Byung-Chull Han	2025-12-18 16:49:55.692768+00	
☐	4	UML	Gilleanes T A Guedes	2025-12-19 12:16:48.335808+00	

Resumo da Aula: Do Local para a Nuvem

➤ A Evolução da Arquitetura

- **Antes (CineApp):** Usávamos AsyncStorage. Os dados ficavam presos no celular e eram perdidos se limpássemos o cache.
- **Agora (BookApp):** Conectamos o App a um **Backend as a Service (Supabase)**. Nossos dados agora vivem em um banco **PostgreSQL** na nuvem, acessível de qualquer lugar.

Resumo da Aula: Do Local para a Nuvem

➤ O Ciclo do CRUD Completo

- Hoje implementamos as 4 operações fundamentais de qualquer sistema comercial:
- **CREATE (.insert)**: Cadastramos novos livros no banco.
- **READ (.select)**: Buscamos a lista atualizada e ordenamos por ID.
- **UPDATE (.update)**: Alteramos dados existentes (filtrando pelo `.eq('id', ...)`).
- **DELETE (.delete)**: Removemos registros permanentemente.

Resumo da Aula: Do Local para a Nuvem

► Técnicas Avançadas de React Native

- **Reutilização de Telas:** Aprendemos a usar a mesma tela (BookForm) para **Criar** e **Editar**, usando a lógica: "*Se tem ID, edita; se não tem, cria*".
- **Gestão de Foco (useFocusEffect):** Resolvemos o problema de "lista desatualizada" ao voltar de outra tela.
- **Feedback ao Usuário:** Usamos ActivityIndicator (loading) e Alert para garantir que o usuário saiba o que está acontecendo enquanto a internet trabalha.

Resumo da Aula: Do Local para a Nuvem

- **O Desafio da Segurança (RLS)**

- Entendemos que bancos de dados reais vêm "trancados" por padrão.
- Aprendemos a manipular o **Row Level Security (RLS)** do Supabase para permitir ou bloquear o acesso aos dados.

Fim



Prof. Maromo